30-Day Roadmap Full Stack Developer Intern

Target: Full Stack Developer Intern (and junior roles — apply to both)
Stack: Python Django + DRF PostgreSQL HTML/CSS/Tailwind JavaScript React Deployment
Background assumed: SE graduate — you know OOP, basic DSA, and DB theory already
Daily budget: 7–8 hours

> *This replaces the backend-only roadmap. The Django track is now shallower (advanced ORM, Celery, and deep testing are cut) to make room for frontend. For an intern application that's the right trade.*

What interns are actually judged on

Weight	What
High	CS fundamentals, DSA, live coding
High	One working, deployed full-stack project you can explain
High	Coachability, communication, curiosity
Medium	Git hygiene, clean code
Low	Production concerns (Celery, caching, scaling, system design)

Interns get hired on *potential*. Juniors get hired on *proof*. You're being evaluated on how fast you'll learn, so the way you talk about your project matters as much as the project.

Daily time split

Block	Hours	What
Learn	3	New concepts — docs, video, reading
Build	3–4	Write code from a blank file
Revise	1	Notes, yesterday's topic, 2 DSA problems

Start the **2 DSA problems per day** habit on Day 1. Don't save it all for week 4.

WEEK 1 — Python + the visual layer

Day 1 — Python core

- Types, mutability (interview favourite), `list` / `tuple` / `dict` / `set` and their complexities
- Slicing, negative indexing, unpacking, string methods, f-strings
- List/dict/set comprehensions, `enumerate`, `zip`, `sorted(key=...)`
- **Build:** 15 small problems — word frequency, dedupe, flatten a nested list, group by key

Day 2 — Functions & structure

- Default args (and the mutable default trap), `*args` / `**kwargs`
- Scope, closures, decorators — write 3 yourself (timer, logger, retry)
- Modules, packages, imports, `venv`, `pip`, `requirements.txt`
- **Build:** a decorator that caches an expensive function's results

Day 3 — OOP, errors, files

- Classes, `super()`, inheritance vs composition
- Dunders: `__str__`, `__repr__`, `__eq__`, `__len__`
- `@property`, `@classmethod`, `@staticmethod`
- Exceptions: try/except/else/finally, custom exception classes
- Context managers (`with`), file I/O, `json`, `pathlib`, type hints, `dataclasses`
- **Build:** a small library/inventory system with custom exceptions and JSON persistence

Day 4 — Git, terminal, and Project 0

- Git: branch, merge, rebase basics, stash, resolving a conflict on purpose
- GitHub: remotes, pull requests, `.gitignore`, commit conventions (`feat:`, `fix:`)
- Terminal: `cd`, `ls`, `grep`, `find`, pipes, env vars
- **Build:** finish Day 3's project as a proper repo — folder structure, `requirements.txt`, real README, clean commit history

Day 5 — HTML + CSS

- Semantic HTML: `header`, `nav`, `main`, `section`, `article`, `form`, `table`
- Forms and inputs properly (labels, types, validation attributes)
- Box model, `display`, `position`, specificity, the cascade
- **Flexbox** — learn this until it's automatic (Flexbox Froggy)
- **Build:** a static landing page from scratch, no framework

Day 6 — CSS Grid, responsive, Tailwind

- CSS Grid (Grid Garden), when Grid beats Flexbox
- Media queries, mobile-first, `rem` / `em` / `%` / `vh`
- Tailwind CSS: utility classes, responsive prefixes, why utility-first
- **Build:** a responsive dashboard layout — sidebar, topbar, card grid — in Tailwind

Day 7 — JavaScript fundamentals I

- `let` / `const`, types, coercion, `==` vs `===`
- Array methods: `map`, `filter`, `reduce`, `find`, `some`, `every`, `sort`
- Objects, destructuring, spread/rest, optional chaining
- Arrow functions vs regular functions, template literals
- Scope, hoisting, closures
- **Build:** 15 JS problems using array methods only, no `for` loops

---

WEEK 2 — JavaScript, SQL, and Django

Day 8 — JavaScript fundamentals II

- DOM: `querySelector`, creating/removing elements, `classList`
- Events, event listeners, bubbling, `preventDefault`
- Form handling and client-side validation
- `localStorage`
- **Build:** a to-do app in vanilla JS with localStorage persistence. Do this before React — it makes React make sense.

Day 9 — Async JavaScript (critical)

- The event loop, call stack, callback queue — understand this, it gets asked
- Callbacks Promises `async` / `await`
- `fetch`, request options, headers, parsing JSON
- Error handling: `.catch`, try/catch with await, handling non-200 responses

- **Build:** fetch from a public API (PokéAPI, JSONPlaceholder) with loading and error states rendered in the DOM

Day 10 — SQL

- PostgreSQL setup + DBeaver or pgAdmin
- `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, aggregates
- `JOIN`: INNER vs LEFT — draw them until obvious
- Primary/foreign keys, indexes (what they cost), normalization to 3NF
- **Practice:** 20 HackerRank SQL problems

Day 11 — HTTP & REST

- Request/response cycle, headers, query vs path params
- Methods and idempotency; status codes: 200, 201, 204, 400, 401, 403, 404, 409, 500
- Statelessness, cookies vs tokens, **CORS** (you'll hit this in week 3 — know it now)
- REST resource naming conventions
- Postman: collections, environments, saving auth tokens
- **Build:** design the full endpoint list for your week-4 project on paper

Day 12 — Django core

- `startproject` vs `startapp`, settings, URL routing, the request lifecycle
- Views, `JsonResponse`, models, field types, migrations
- Django admin, `ModelAdmin` config
- **Custom user model** — set this up on day one of every project, and know why
- `.env` config with `django-environ`, secrets never in the repo
- **Build:** a Django project with 3 related models running on PostgreSQL

Day 13 — Django ORM

- QuerySet laziness — when does the DB actually get hit?
- `filter`, `exclude`, `get`, `exists`, `count`; lookups: `__icontains`, `__gte`, `__in`
- `ForeignKey`, `ManyToManyField`, `related_name`, reverse relations
- **The N+1 problem** — `select_related` vs `prefetch_related`. Learn this properly even on the intern track; it's the one "deep" thing that makes you stand out.
- `annotate`, `aggregate`, `Q()` objects
- **Build:** 25 ORM queries against your models, inspecting `.query` each time

Day 14 — Django templates & auth (server-rendered full stack)

- Template syntax, inheritance (`{% extends %}` , `{% block %}`), static files
- Django forms and `ModelForm` , CSRF
- Registration, login/logout, `LoginRequiredMixin` , ownership checks
- **Build:** a complete server-rendered CRUD app with auth. This is genuinely full stack in the Django sense, and it's a real portfolio piece — ship it.

WEEK 3 — DRF + React + wiring them together

Day 15 — DRF serializers & views

- `Serializer` vs `ModelSerializer` , `fields` , `read_only_fields`
- Nested serializers, `SerializerMethodField`
- Validation: field-level, object-level, custom validators
- `APIView` generic views `ViewSet` + routers
- **Build:** full CRUD API for your models

Day 16 — DRF auth & API polish

- JWT with `djangorestframework-simplejwt` : access vs refresh, expiry
- Permissions: built-in + one custom class (`IsOwnerOrReadOnly`)
- Pagination, `django-filter` , search, ordering
- Swagger docs with `drf-spectacular`
- **CORS setup** with `django-cors-headers` — you need this tomorrow
- **Build:** register / login / refresh + protected endpoints, tested in Postman

Day 17 — React fundamentals

- Vite setup (`npm create vite@latest`), project structure
- JSX rules, components, props
- `useState` , event handlers, conditional rendering
- Rendering lists and why `key` matters
- Composition and splitting components sensibly
- **Build:** rebuild your vanilla-JS to-do app in React

Day 18 — React with data

- `useEffect` : dependency array, cleanup, the infinite-loop trap

- Fetching data, loading and error states
- Controlled inputs and forms
- Lifting state up, prop drilling, `useContext`
- **Build:** a React app consuming a public API — search, filter, loading spinner, error message

Day 19 — Routing & structure

- React Router: `BrowserRouter`, `Routes`, `Route`, `Link`, `useNavigate`, `useParams`
- Protected routes and redirects
- Axios, a central API service file, base URL from `.env`
- Folder structure: `components/`, `pages/`, `services/`, `hooks/`
- Custom hooks — write one (`useFetch`)
- **Build:** multi-page React app with routing and a shared layout

Day 20 — Connecting React Django (the day that makes you full stack)

- CORS in practice — see the error, understand it, fix it
- Login flow: POST credentials receive JWT store it attach to requests
- Where to store tokens, and the trade-offs (this is a great interview answer)
- Auth context provider, protected routes gated on auth state
- Axios interceptors: attach token, handle 401, refresh flow
- Handling API errors and displaying them in the UI
- **Build:** working end-to-end login on your own Django API

Day 21 — Polish + basic testing

- Django: `pytest-django`, 10–15 tests on your API endpoints and permissions
- React: form validation, empty states, disabled buttons while submitting, toast notifications
- Responsive check on mobile widths
- **Build:** clean up everything so far — this is the foundation of your main project

---

WEEK 4 — Main project, deploy, apply

Days 22–24 — MAIN FULL STACK PROJECT (3 days)

Pick **one** and build it properly:

- **Task / Project Manager** — projects, tasks, assignees, statuses, comments, roles
- **E-commerce storefront** — products, categories, cart, orders, order history
- **Job board** — listings, applications, employer vs seeker roles, search & filter

Required:

- Django + DRF backend, PostgreSQL, JWT auth, custom user model
- Custom permissions (users only edit their own data)
- Pagination, search, filtering
- React frontend: routing, protected routes, auth context, forms with validation
- Loading states, error states, empty states — juniors always forget these
- Responsive with Tailwind
- 10+ backend tests
- Swagger docs
- Clean commit history across the 3 days, not one giant commit

Day 25 — Deploy it live

- Backend: `DEBUG=False`, `ALLOWED_HOSTS`, env vars, WhiteNoise for static deploy to **Railway** or **Render** with managed PostgreSQL
- Frontend: build deploy to **Vercel** or **Netlify**, point it at the live API
- Fix the CORS and env-var issues that will absolutely appear
- Docker basics: `Dockerfile`, `docker-compose` for Django + Postgres (concept level is fine for an intern)
- **Deliverable:** a live URL you can put on your resume

Day 26 — Resume, GitHub, LinkedIn + START APPLYING

- **GitHub:** pin 3 repos. Each needs a README with — what it does, stack, live demo link, setup steps, screenshots or a GIF. Archive the junk.
- **Resume (1 page):** Skills Projects (with live links) Education coursework/FYP. Put the stack in the top third where a recruiter's eye lands.
- **LinkedIn:** headline "Full Stack Developer | Django, React, PostgreSQL", About section, featured projects, open-to-work.
- **Send your first 15 applications today.** Not on Day 30.

Day 27 — DSA I

- Big-O of everything you write
- Arrays & strings: two pointers, sliding window

- Hash maps: frequency counting, seen-sets
- **20 LeetCode Easy** in Python — out loud, explaining as you go

Day 28 — DSA II + SQL

- Sorting, binary search, stacks (valid parentheses)
- Recursion basics
- **10 LeetCode Easy + 5 Medium**, plus 10 SQL problems
- Practice writing code in a plain editor with no autocomplete — that's the interview condition

Day 29 — Mock interviews

- Work through the question bank below **out loud**, timed
- Record a 3-minute explanation of your project: problem architecture hardest part what you'd do differently
- Behavioural: tell me about yourself, a time you were stuck, why this company, where you want to be in 2 years
- Prepare 3 questions for them (mentorship, what an intern's first month looks like, code review process)

Day 30 — Volume

- 20 more applications
- Ongoing rhythm until hired: 2 DSA problems + 1 hour building, daily

---

Interview question bank

JavaScript

1. `var` / `let` / `const` — differences and hoisting behaviour
2. `==` vs `===`
3. What is a closure? Give a real use
4. Explain the event loop
5. Promises vs `async` / `await` — and how to handle errors in each
6. `map` vs `forEach` vs `filter` vs `reduce`
7. Spread vs rest
8. Shallow vs deep copy in JS
9. What is `this` and how does it change?

10. Synchronous vs asynchronous — why does JS need async at all?

React

1. What is JSX and what does it compile to?
2. Virtual DOM and reconciliation
3. Props vs state
4. Why do lists need `key`? What's wrong with using the array index?
5. `useEffect` dependency array — what happens with `[]`, with `[x]`, with nothing?
6. What causes a re-render?
7. Controlled vs uncontrolled components
8. Prop drilling and how Context solves it
9. When would you write a custom hook?
10. How do you handle loading and error states when fetching?

Python

1. Mutable vs immutable, with a gotcha
2. List vs tuple vs set vs dict, and their complexities
3. Shallow vs deep copy
4. Decorators — write one live
5. Generators vs lists
6. `*args` and `**kwargs`
7. `is` vs `==`
8. `@staticmethod` vs `@classmethod`
9. Mutable default argument problem
10. How does Python handle memory?

Django / DRF

1. Explain MTV and the request lifecycle
2. What is middleware?
3. `select_related` vs `prefetch_related`, and the N+1 problem
4. QuerySet laziness
5. `Serializer` vs `ModelSerializer`
6. How does JWT auth work? Access vs refresh tokens
7. Where do you store a JWT on the frontend, and what are the trade-offs?
8. How do you write a custom permission class?
9. What is CSRF, and why don't JWT APIs need it?
10. Why a custom user model from the start?
11. Why Django over Flask/FastAPI for your project?

SQL & general

1. INNER vs LEFT JOIN
2. What is an index and when does it hurt?
3. Normalization to 3NF
4. `WHERE` vs `HAVING`
5. What is ACID?
6. Explain REST
7. PUT vs PATCH
8. What is CORS and why did it break your app?
9. Walk me through what happens between typing a URL and seeing the page
10. Git: how do you resolve a merge conflict?

Skip these (protects your month)

- TypeScript — valuable, but not in 30 days
- Redux / Zustand — `useContext` is plenty for an intern
- Next.js — plain React + Vite is fine
- Celery, Redis, Kubernetes, microservices, GraphQL
- FastAPI or Flask — one backend framework, known well
- React testing libraries
- Angular, Vue
- Ten tutorial projects instead of two real ones

Application strategy for internships

- **Apply to intern *and* junior roles.** Same resume. You're a graduate, you're eligible for both, and self-limiting costs you offers.
- Many software houses run **structured paid internship cohorts** with intake windows — find those windows and hit them. Conversion to full-time is the real prize, not the stipend.
- **Channels:** LinkedIn Jobs with alerts, company career pages directly, Indeed. If you're in Pakistan, Rozee.pk plus emailing HR directly — direct application often beats portals.
- **Cold outreach genuinely works for interns.** Three lines to an engineering manager or HR on LinkedIn: who you are, your best project with a live link, what you're looking for. Send 5 a day.

- **Use your university.** Career office, faculty referrals, alumni working at target companies. Massively underused and the highest-conversion channel available to a fresh graduate.
- Track in a spreadsheet: company, role, date, contact, status.
- Target **50+ applications** across the month.
- Ask for feedback after rejections. Some give it, and it's free calibration.

Honest expectations

In 30 days you will be a **credible full stack intern candidate** — someone who has built and deployed a real application and can explain it clearly. You will not be a full stack developer yet, and you shouldn't claim to be. That gap is exactly what an internship is for, and interviewers respond well to a candidate who knows what they don't know.

Two things decide your outcome: **how well you can explain your own project**, and **how many applications you actually send**. Almost everyone underperforms on the second one.